

Turnitin Check

acis.docx

-  Article Plagiarism
-  My Task
-  Manipal Academy of Higher Education (MAHE)

Document Details

Submission ID

trn:oid:::1:3628906956

Submission Date

Aug 17, 2026, 11:58 PM GMT+8

Download Date

Aug 18, 2026, 12:00 AM GMT+8

File Name

acis.docx

File Size

32.9 KB

11 Pages**3,118 Words****19,362 Characters**

39% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



20 AI-generated 39%

Likely generated or likely generated and revised by AI.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Autonomous Credit Intelligence: An Event-Driven, Self-Healing Multi-Agent Framework for Real-Time B2B Risk Assessment

Nikhil Nagendra

Pallavi K V

Bhoomi Joshi

Johan K George

Nitin Bharadwaj MVS

Prasanna B Acharya

Batch reviews of B2B accounts receivable still miss same-day credit deterioration because payment behaviour, financial fundamentals, and litigation signals are often scored in separate cycles. This paper presents ACIS-X (Autonomous Credit Intelligence System, Extended), an event-driven multi-agent framework on Apache Kafka that maintains a live per-customer risk score. Specialised agents exchange only Kafka events across five processing layers plus a system plane: behavioural metrics, external enrichment and three-tier fusion, payment prediction with SHAP audit persistence, risk refinement, policy and collections actions, durable storage, and heartbeat-based self-healing that resumes consumers from the last committed offset. We evaluate the design with 162 automated tests (unit, integration, self-healing, and performance), including live-broker discovery. Because production AR ledgers are commercially sensitive, predictive comparisons use a seeded scenario generator rather than a bank default book. On the evaluated path, pipeline P95 latency is 16 ms, risk-scoring throughput scales to $3.49 \times$ at four workers, and multi-signal refinement outperforms a utilisation-only rule on synthetic data.

Introduction

Management of enterprise credit risks calls for constant evaluation of payment status, financial position, and potential litigations. Daily or weekly batch cycles delay action when risk rises within a day. Event streaming platforms such as Apache Kafka and multi-agent designs for credit assessment make it practical to keep a live credit view instead of an overnight refresh.

This work introduces ACIS-X (Autonomous Credit Intelligence System, Extended), a Kafka-based multi-agent system for B2B credit intelligence. It targets three gaps in prior practice: (1) latency of batch monitoring; (2) siloed behavioural, financial, and litigation signals; and (3) fragile pipelines without autonomous recovery.

The contributions are: a five-layer Kafka-only agent architecture with a cross-cutting self-healing plane; a three-tier fusion path from behavioural metrics, external financials,

and litigation into one score in $[0,1]$; heartbeat monitoring with restart from committed offsets; and an offline-plus-live test suite (162 tests) that measures pipeline latency, deduplication, scaling, and predictive quality on synthetic data.

Related Work

Classical credit scoring such as Altman’s Z-score and recent explainable ML for bank/supply-chain credit remain tied to offline or periodic feature tables, so they are a poor fit for intraday AR monitoring even when SHAP improves transparency. Ensemble and hybrid learners raise accuracy in published studies, yet most refresh on batch folds rather than continuous invoice streams. In parallel, Kafka-based event sourcing and Kafka–Flink pipelines have moved financial risk and fraud analytics onto high-throughput streams, which shows that sub-second paths are achievable, but those systems usually score transactions or market exposures rather than a standing B2B credit profile that also pulls fundamentals and litigation.

Multi-agent credit work is emerging: MASCA uses hierarchical LLM agents for credit assessment, with reasoning and fairness analysis rather than an operational Kafka choreography. Self-healing ideas go back to autonomic MAPE-K control; recent streaming microservices add circuit breakers, buffers, and replay patterns, often restarting containers or replaying topics without agent-local heartbeat timeouts. Table 1 summarises capability presence only (stream path, AR fusion, self-healing, litigation, regression suite) for three prior lines of work versus ACIS-X; cells are qualitative checkmarks, not shared-benchmark scores, because those systems are not runnable on our AR generator. ACIS-X sits at the overlap of streaming credit fusion and fine-grained agent recovery: agents exchange only Kafka events, fuse three signal families online, and resume from the last committed offset after heartbeat timeouts.

Qualitative positioning (capability presence, not shared-benchmark scores)

Capability	Batch XAI	Stream risk	MASCA	ACIS-X
Real-time stream path	×		×	
Multi-signal AR fusion	×	×	partial	
Self-healing + offset resume	×	×	×	
Litigation signals	×	×	×	
Automated regression suite	×	×	×	162 pass

System Architecture

ACIS-X maintains a live risk score for each customer c_i :

$$R(c_i, t) = F(B(c_i, t), E(c_i, t), L(c_i, t)),$$

where B is behavioural history, E external financial indicators, and L litigation exposure. F maps those inputs to $[0,1]$. Design targets are sub-second processing on the evaluated path, usable scores under partial signals, no silent loss of committed work across restart, and idempotent duplicate handling.

Figure [fig:arch] is the architecture map of that runtime. Layers 1–4 sit above a shared Apache Kafka event bus; Layer 5 (persistence) and the system plane sit below. Boxes connect only to Kafka, not peer-to-peer: Layer 1 builds customers, invoices, payments, and behavioural metrics; Layer 2 enriches with financial APIs and litigation, then aggregates; Layer 3 predicts payment risk and refines the composite score; Layer 4 applies policy and collections actions; Layer 5 persists state; the system plane runs time ticks, monitoring, self-healing, DLQ watching, and runtime placement.

Agent Inventory

Each agent has one job so failures stay local and Kafka remains the only integration surface. Table 2 lists every processing agent with its layer and a one-line purpose (what it consumes conceptually and why it exists); system-plane roles are summarised immediately under the table. Subscriptions follow the layer flow in Fig. [fig:arch].

Processing agents and why each exists

Agent	L	Purpose
Scenario generator	1	Emits synthetic customers, invoices, and payments for demos and reproducible tests (not a production ERP feed).
Customer state	1	Rebuilds per-customer behavioural metrics (on-time ratio, delays, utilisation) from invoice/payment events.
External data	2	Pulls/caches financial ratios (debt/equity, ROE, PE, growth) onto the metrics stream.
Litigation scraping	2	Scores court/news litigation exposure and publishes it as a metrics enrichment.
Aggregator	2	Fuses financial and litigation risks into one external profile on the risk topic.

Agent	L	Purpose
Payment prediction	3	Scores open invoices with RF + hard policy overrides; stores SHAP attributions for audit.
Risk scoring	3	Refines the base score with behavioural, external, and temporal adjustments into the final <i>R</i> .
Overdue detection	4	On each time tick, marks unpaid past-due invoices so downstream agents see freshness of delinquency.
Credit policy	4	Maps risk bands to control intents (watch, hold, escalate) for credit operations.
Collections	4	Turns intents into idempotent collection actions (reminders/escalations) without duplicate storms.
Database	5	Upserts all topic events into SQLite/Postgres so state survives process restarts.
Query / memory	5	Serves hot reads: SQL-backed queries plus a locked in-memory snapshot for low-latency lookups.

Table [tab:agent-topics] complements the purpose listing above with the concrete Kafka topic wiring derived from the implementation codebase: which topics each agent subscribes to and which it publishes to.

Table 3. ACIS-X Agent Inventory – Subscriptions and Publications

Agent	Layer	Subscribes To	Publishes To
ScenarioGeneratorAgent	1	acis.commands	customers, invoices, payments
CustomerStateAgent	1	invoices, payments	acis.metrics

Agent	Layer	Subscribes To	Publishes To
ExternalDataAgent	2	acis.metrics	acis.metrics (enriched)
ExternalScrapingAgent	2	acis.metrics	acis.metrics (litigation)
AggregatorAgent	2	acis.metrics	acis.risk
PaymentPredictionAgent	3	acis.metrics	acis.predictions
RiskScoringAgent	3	predictions, customers, metrics, risk	acis.risk
OverdueDetectionAgent	4	acis.time	acis.invoices
CreditPolicyAgent	4	acis.risk	acis.collections
CollectionsAgent	4	acis.collections	acis.collections
DBAgent	5	ALL topics	SQLite (persist)
QueryAgent / MemoryAgent	5	(synchronous calls)	(in-memory cache)

System plane. Time tick publishes a periodic clock for overdue checks; monitoring watches heartbeats; self-healing requests restart and offset resume after timeout; the DLQ monitor surfaces poison messages; the runtime manager places/restarts agent processes and supports dynamic topic discovery.

Events are keyed by customer id so one customer stays ordered. Agents use dedicated consumer groups; topics have three partitions. On start, agents register and bind to topics dynamically (checked by live-broker discovery tests). Local runs use SQLite; Postgres is used when a database URL is configured.

Methodology

Three-Tier Signal Fusion

In plain terms, Tier 1 measures how the customer pays; Tier 2 adds outside financial health and court exposure; Tier 3 mixes those external scores.

Tier 1 – Behavioural signals. The customer state agent rebuilds metrics from invoices and payments under a per-customer lock. Let $I(c_i)$ be the invoice set of customer c_i , $I_k \in I(c_i)$ one invoice, and P the payment set used for delay. $paid(I_k)$ and $due(I_k)$ are payment and due timestamps; $paid_k$ and due_k are the same quantities for payment k . The *on-time ratio* (OTR) and average delay are

$$OTR(c_i) = \frac{|\{I_k: paid(I_k) \leq due(I_k)\}|}{|I(c_i)|},$$

$$AvgDelay(c_i) = \frac{1}{|P|} \sum_k \max((paid_k - due_k), 0).$$

OTR is a hit rate in $[0,1]$; AvgDelay is non-negative days. Both feed Layer 3 features (via normalised delay and late-payment rate).

Tier 2 – External signals. The external data agent caches debt/equity, ROE, PE, and growth for 90 days and maps them to a financial risk $E(c_i) \in [0,1]$. The litigation scraping agent scores court exposure over a 365-day window as $L(c_i) \in [0,1]$. Both publish onto the metrics topic.

Tier 3 – Fusion. The aggregator agent combines those external scores as

$$R_{\text{ext}}(c_i) = 0.6 \cdot E(c_i) + 0.4 \cdot L(c_i) \in [0,1].$$

The 0.6/0.4 split is a fixed heuristic: fundamentals are treated as the primary external signal and litigation as a severe but sparser supplement. Weights were chosen for interpretability and to keep behavioural path dominant downstream; they were *not* grid-searched on a labelled default book. Stale events are dropped; a risk-profile update is published only when fused values change.

Payment Risk Prediction

The payment prediction agent scores each open invoice. Let $X \in R^d$ be the feature vector (normalised delay $s_{\text{delay}} \in [0,1]$, credit utilisation, invoice-to-limit ratio, late-payment rate, external/financial/litigation terms, and a rating penalty). A Random Forest of $M = 50$ trees yields base probability r_{ml} ; $h_m(X)$ is the m -th tree's vote in $[0,1]$:

$$r_{ml} = \frac{1}{M} \sum_{m=1}^M h_m(X), r_{\text{final}} = \max(0, \min(1, r_{ml} + r_{\text{policy}})).$$

Here r_{policy} is a hard override sum: rating penalties (+0.10 for B, +0.20 for C) plus litigation bumps (+0.15 if $L > 0.1$, +0.25 if $L > 0.5$). These additives encode non-negotiable credit rules that must not be diluted by the forest. The forest is *warmed up at process start on 1,000 synthetic feature rows* (not a labelled bank default portfolio). The final score is clamped to $[0,1]$.

For audit we store SHAP attributions. With model f , baseline $E[f(X)]$, and per-feature contribution φ_j for feature $j = 1, \dots, d$,

$$f(X) = E[f(X)] + \sum_{j=1}^d \varphi_j.$$

Policy additives sit outside this decomposition and are logged beside the SHAP vector.

Risk Refinement and Final Scoring

The risk scoring agent starts from the prediction r_{pred} (i.e. r_{final} or the latest scored risk) and forms

$$r_{\text{adj}} = r_{\text{pred}} + 0.85 \cdot \Delta_{\text{behavioral}} + 0.30 \cdot \Delta_{\text{temporal}},$$

then clamps r_{adj} to $[0,1]$ (high 0.75, medium 0.40, else low). The blend 0.85 lets present behavioural evidence move the score enough to separate clean vs delinquent accounts; 0.30 caps temporal influence so velocity cannot dominate the level. Both coefficients are engineering choices validated by suite behaviour, not maximum-likelihood estimates.

Behavioural evidence is a weighted sum of normalised factors $a. \in [0,1]$:

$$\begin{aligned} \Delta_{\text{behavioral}} = & 0.40a_{\text{overdue}} + 0.20a_{\text{payment}} \\ & + 0.15a_{\text{delay}} + 0.15a_{\text{outstanding}} + 0.10a_{\text{exposure}}. \end{aligned}$$

Meanings: a_{overdue} overdue intensity; a_{payment} late-pay behaviour; a_{delay} delay severity; $a_{\text{outstanding}}$ utilisation pressure; a_{exposure} large-balance pressure. Weights sum to 1 and rank operational urgency (missed due dates first, then payment pattern, then size/exposure). Optional external context blends aggregated/financial/litigation/market terms with weights 0.40/0.30/0.20/0.10 and may add at most about 20% on top of $\Delta_{\text{behavioral}}$, so outside data amplifies rather than replaces payment evidence.

Temporal term Δ_{temporal} comes from a risk-velocity query: positive bumps for fast/slow deterioration and high volatility, a small negative bump when the trend is improving, then clamped to $[-0.2,0.3]$ before the 0.30 mix in [eq:refinement].

Self-Healing Protocol

Agents emit heartbeats every $T_H = 10$ s. Monitoring marks a timeout after 60 s without a heartbeat (six missed beats, chosen to tolerate brief GC/IO stalls). Self-healing then emits a restart request; the runtime resumes the consumer from the last committed offset. Time ticks (system plane) fire on a short interval so overdue detection stays wall-clock driven even when invoice traffic is quiet.

Implementation

The system is implemented in Python 3.11 across approximately 50 source files organised into six packages: agents, runtime, self-healing, monitoring, registry, and schemas. A single entry point pre-creates Kafka topics, then launches every component as an independent OS process. A supervisor restarts crashed agents but caps restarts at five within two minutes; beyond that limit the agent is marked permanently failed and a critical alert is published to Kafka. Every message conforms to a Pydantic-validated event envelope (schema v1.1) carrying an event id, type, source, timestamp, entity id, payload, and a correlation id for end-to-end tracing. Messages that fail validation are routed to a per-topic dead-letter queue.

The broker layer runs three KRaft-mode nodes (Confluent Platform 7.5, replication factor 3, ISR 2) with sixteen pre-created topics—six business-event, four system, three auxiliary, and corresponding dead-letter topics. All events are keyed by customer or agent id to preserve ordering; each agent uses a dedicated consumer group with explicit offset commits. Idempotency is enforced at three levels: an in-memory ordered map of the last 10 000 event ids ($O(1)$ dedup), primary-key upserts in the database agent, and application-level guards (e.g. duplicate-action checks in collections, change-detection before republishing fused risk). The SQL store supports SQLite locally and Postgres (Supabase) in production, with dialect-agnostic SQL and automatic schema migrations on startup. Concurrency is managed through per-customer and per-component locks rather than a global lock.

Four resilience patterns complement the self-healing protocol: a circuit breaker on external API calls (opens after five failures, probes after 60 s), a retry module with exponential backoff and jitter (budget exhaustion routes to the dead-letter queue), consumer-loop fault tolerance that defers rebalance-related commit errors, and a control topic that allows runtime pause/resume of any agent. Docker Compose orchestrates the three-node Kafka cluster, the ACIS engine, a FastAPI gateway, and an Nginx-served React dashboard; cloud deployments on Confluent Cloud and Supabase Postgres are supported via an environment-file overlay.

Experimental Evaluation

Test Infrastructure

We re-ran the project suite on Windows 11 (i7-12700H, 32 GB), with Docker Kafka healthy: **162 passed**, 0 skipped. Most cases use mocked Kafka/HTTP; dynamic discovery also passed against the live broker. Latency and scaling figures below are in-process measurements unless noted.

Systems Metrics

Table 3 reports systems metrics from the automated suite on the evaluated path: pipeline P95 latency, collections deduplication under repeated events, risk-scoring throughput scaling with worker count, and overall pass count. These are regression guards from mocked or in-process runs unless noted, not cluster SLAs.

Systems metrics from the automated suite

Metric	Value	Source
Pipeline P95 (mocked, $n = 500$)	16 ms (mean 4.6 ms)	performance suite
Collections dedup under 10k repeats	1 action	performance suite
Risk-scoring scaling	$3.49 \times$ at $N = 4$ (peak $4.93 \times$ at $N = 6$)	throughput test
Suite result	162/162 pass	full test suite

Predictive Comparison on Synthetic Data

Quantitative comparisons in this paper are *reproducible baselines on the same synthetic generator*, not re-implementations of Chang et al., Parnerkar, or MASCA (those target different tasks/data and are not drop-in substitutes for invoice-level B2B AR streams). We use an in-process scenario generator rather than a production AR ledger because B2B invoice and payment histories are commercially sensitive and rarely releasable for open evaluation; public credit corpora are also a poor fit for continuous invoice-level streams with litigation enrichment. The generator lets us fix seeds, inject overdue and external-risk patterns, and re-run the full suite reproducibly. All predictive numbers below therefore come from *synthetic* customers and RF warm-up rows; they are not a bank default ledger.

Baselines compared. (i) a naive utilisation-threshold rule; (ii) single-signal rankers (AR-only, payment behaviour, overdue-only) via Spearman ρ ; (iii) supervised LR / Random Forest / Gradient Boosting trained on the same labelled hold-out fold. ACIS-X risk scoring is evaluated as an online multi-factor refiner *without* fitting those labels.

Streaming refinement vs a rule baseline (n=100, suite). ACIS-X risk scoring (multi-factor refinement) achieved F1 0.824 versus a utilisation-threshold rule F1 0.653. Spearman rank correlation with ground-truth risk was $\rho = 0.719$ for ACIS-X, against 0.648 (AR-only), 0.550 (payment behaviour), and 0.418 (overdue-only).

Supervised baselines (n=500, 70/30 split, seed 42). We trained Logistic Regression, Random Forest ($M = 50$), and Gradient Boosting on the same generator (XGBoost was not installed; GBM is the boosted-tree baseline). Features are utilisation, normalised delay, late rate, and overdue intensity; the enriched setting adds financial and litigation risks. Table 4 is the hold-out scorecard: F1 and AUC for the utilisation rule, untrained ACIS-X risk scoring, and each supervised learner in basic vs enriched feature settings. Enrichment improves every learner. Supervised models, which fit the labelling function, outscore the untrained streaming refiner on this generator—as expected. ACIS-X still beats the naive utilisation rule on the same hold-out fold (F1 0.787 vs 0.548) without training on labels, which matches its role as an online rule layer rather than a batch classifier.

Quantitative comparison on one synthetic hold-out (n=500, seed 42). Enriched = behavioural + external features.

Method	F1	AUC
Naive utilisation threshold	0.548	–
ACIS-X risk scoring (no label training)	0.787	0.820
Logistic Regression (basic)	0.944	0.991
Logistic Regression (enriched)	0.994	1.000
Random Forest (basic)	0.893	0.963

Method	F1	AUC
Random Forest (enriched)	0.938	0.980
Gradient Boosting (basic)	0.938	0.976
Gradient Boosting (enriched)	0.950	0.991

Discussion

External enrichment improves both supervised F1 and streaming rank correlation on the synthetic population. The scenario generator was a deliberate evaluation choice for privacy and reproducibility, not a claim that synthetic labels replace a real default book; partner data remains future work. Fusion and refinement weights are documented heuristics (behavioural evidence first; external/temporal as bounded supplements), not fitted default probabilities. The agent table and Kafka layout make the choreography readable, but end-to-end tracing still needs span ids in production. SQLite suits local demos; Postgres is the shared path. Self-healing tests cover decision correctness and storm prevention; full process failover timing needs a live rebalance harness. Limits: synthetic labels and RF warm-up are not a real default book; P95/scaling are regression guards, not cluster SLAs.

Conclusion

ACIS-X is a Kafka multi-agent credit pipeline with three-tier fusion, SHAP audit rows, and heartbeat-driven recovery. On the suite re-run, all 162 tests passed; measured P95 was 16 ms; scaling reached $3.49 \times$ at four workers. On synthetic data, multi-signal refinement beats a utilisation rule, and feature enrichment lifts LR/RF/GBM baselines. Future work: labelled evaluation sets and live-broker MTTR drills.

Acknowledgement

The authors thank the faculty and laboratory infrastructure of the Department of Computer Science and Engineering, AMC Engineering College, Bengaluru, for supporting this research.

10

Kreps, J., Narkhede, N., Rao, J.: Kafka: A distributed messaging system for log processing. In: Proc. NetDB (2011)

Jajoo, G., Pandey, A., Chitale, P., Agarwal, S.: MASCA: LLM-based multi-agent system for credit assessment. arXiv preprint arXiv:2507.22758 (2025)

Altman, E.I.: Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. J. Finance **23**(4), 589–609 (1968)

Chang, V., Xu, Q.A., Akinloye, S.H., Benson, V., Hall, K.: Prediction of bank credit worthiness through credit risk analysis: an explainable machine learning study. *Ann. Oper. Res.* **354**, 247–271 (2025). <https://doi.org/10.1007/s10479-024-06134-x>

Nwafor, C.N., Nwafor, O., Brahma, S.: Enhancing transparency and fairness in automated credit decisions: an explainable novel hybrid machine learning approach. *Sci. Rep.* **14**, 25174 (2024). <https://doi.org/10.1038/s41598-024-75026-8>

Lundberg, S.M., Erion, G., Chen, H., et al.: From local explanations to global understanding with explainable AI for trees. *Nat. Mach. Intell.* **2**(1), 56–67 (2020). <https://doi.org/10.1038/s42256-019-0138-9>

Kesarpu, S., Dasari, H.P.: Kafka event sourcing for real-time risk analysis. *Int. J. Comput. Exp. Sci. Eng.* **11**(3) (2025). <https://doi.org/10.22399/ijcesen.3715>

Parnerkar, H.: Comparative study of event-driven architectures in financial systems for real-time risk analysis and mitigation. *Eur. J. Electr. Eng. Comput. Sci.* **9**(6), 46–54 (2025). <https://doi.org/10.24018/ejece.2025.9.6.761>

Kephart, J.O., Chess, D.M.: The vision of autonomic computing. *Computer* **36**(1), 41–50 (2003). <https://doi.org/10.1109/MC.2003.1160055>

Beerreddy, K.R.: Self-healing streaming architecture: resilience patterns for event-driven microservices. *J. Comput. Sci. Technol. Stud.* **7**(8), 254–268 (2025). <https://doi.org/10.32996/jcsts.2025.7.8.30>